CMPS2131 · DATA STRUCTURES

Lab 0 - C23 Survival Guide

Modern C23 toolchain, memory, pointers, structs, and program organization

Student	Zabdie Ayitor Ayiyi
Date	August 23rd, 2026
Repository / folder	https://github.com/SeeTheLightX/labs

Purpose **This is not a full course in C. It is a survival lab: learn enough modern C23 to compile, inspect, debug, and reason about the programs you will use to study data structures.**

Learning outcomes

- Compile a C23 program with strict warnings and sanitizers enabled.
- Use loops, conditionals, functions, arrays, and pointers correctly.
- Explain the pointer + length contract used when arrays are passed to functions.
- Explain the contract of a null-terminated C string and safely use strlen() and strcmp() on valid strings.
- Separate declarations and implementations using .h and .c files.
- Create structs, use typedef, and access a struct through a pointer.
- Allocate and release dynamic memory, check allocation failure, and use C23 nullptr for null pointers.
- Explain what a self-referential struct stores and why it matters for linked data structures.
- Build a multi-file C23 program with a simple Makefile.

The rule for this course

C23, warning-clean, sanitizer-clean. **A program that "runs" is not automatically a correct program. During development, your code must compile with the required diagnostics and run without sanitizer errors.**

Throughout this lab, use the following development command unless an exercise tells you otherwise:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Wformat=2 -Wvla -Werror -g3 -O1 -fno-omit-frame-pointer \
    -fsanitize=address,undefined hello.c -o hello
```

If you use Clang, replace gcc with clang. Do not remove a warning flag merely to make a diagnostic disappear; understand and correct the code.

Modern C23 null-pointer convention. **Write nullptr whenever you mean "no pointer." You may encounter NULL, integer 0, or (void *)0 in older C code, but do not use those spellings for null pointers in new CMPS2131 code.**

Part 1 - First clean C23 build

Create a folder named lab1 and, inside it, create hello.c.

```
#include <stdio.h>

int main(void)
{
    puts("Hello, C23.");
    return 0;
}
```

1. Compile the program using the required development command.

2. Run it with ./hello.
3. Change main(void) to main(), compile again, and observe that it is still a no-parameter function in C23. Restore main(void); this course style uses the explicit form to make the contract obvious.
4. Introduce a simple warning (for example, declare a local variable and do not use it). Observe what -Werror does. Then fix the program.

☐ My program compiles with zero warnings and runs successfully.

Checkpoint A - Explain before moving on

• What does -std=c23 control?

Enforces the C23 standard rules

• What is the difference between a compiler warning and a compiler error?

A warning indicates potentially problematic code that still compiles into an executable; an error violates language syntax and stops compilation.

• Why might a course deliberately convert warnings into errors?

To train you to write clean, unambiguous code from the beginning and prevent subtle bugs or undefined behavior from slipping into production binaries unnoticed. (Just let me hardcode pls)

• What do AddressSanitizer and UndefinedBehaviorSanitizer help you detect?

AddressSanitizer helps detects memory safety errors at runtime and UndefinedBehaviorSanitizer helps detects operations that violate C runtime rules

Part 2 - Loops, conditionals, and functions

Create basics.c. Complete the following requirements without using global variables.

5. Write a loop that prints the integers 1 through 10.
6. Inside the loop, use a conditional to print whether each number is even or odd.
7. Create an int square(int value) and use it to print the square of each number.
8. Create bool is_even(int value) and use the function in your conditional. In C23, bool is a language keyword; do not include <stdbool.h> merely to obtain bool.

> Contract question **What must the caller provide to square()? What does square() promise to return? Begin thinking of every function as a small contract between caller and implementation.**
>
> **The caller must provide a valid int argument (within bounds to prevent overflow), and square() will return the mathematical square of that value as an int without causing any side effects.**

☐ basics.c compiles warning-clean and sanitizer-clean.

Part 3 - Arrays are data; pointers do not carry lengths

Create arrays.c. Start with this array:

```
int values[] = {4, 7, 1, 9, 3};
size_t count = sizeof values / sizeof values[0];
```

Write the function below and call it from main:

```
int sum_array(const int *values, size_t count);
```

9. Use size_t for indexes and counts.
10. Do not hard-code the number 5 inside sum_array().
11. Print the count, each element, and the final sum.
12. Change the array to contain seven elements. Your function must continue to work without modification.

Mental model **When an array is passed to a function, the function receives a pointer to elements - not the array length. The caller must provide the length separately. The pointer + length pair is part of the function contract.**

Checkpoint B - Reason about memory

• Why is size_t a better type than int for an array length?

`size_t` is an unsigned integer guaranteed to be large enough to hold the maximum size of any object in memory, preventing overflow on large arrays.

• Can sum_array() discover how many elements are available from values alone?

No. A pointer only holds a memory address, not array metadata or length.

• What does const promise about the elements as viewed through values?

It guarantees `sum_array()` will not modify the array elements through that pointer.

• What could happen if the count is larger than the number of valid elements?

Out-of-bounds memory read, leading to garbage values or a buffer overflow crash (detected by AddressSanitizer).

Part 4 - Pointers: addresses and indirection

Create pointers.c. Use the following starting point:

```
int score = 10;
int *score_ptr = &score;
```

13. Print score.
14. Print the address stored in score_ptr using %p and cast the pointer to (void *).
15. Print the value reached through score_ptr.
16. Change score to 25 by writing through score_ptr.
17. Print score again and explain why it changed.

Dereferencing `*score_ptr = 25` writes directly to the memory address where `score` is stored.

Required form for printing a pointer:

```
printf("%p\n", (void *)score_ptr);
```

Do not memorize “pointer syntax” in isolation. **A pointer is a value that stores an address. Dereferencing follows that address to an object. The syntax makes more sense when you keep the memory model in view.**

Part 5 - C strings are a memory convention

Create strings.c. This exercise is intentionally small. You are learning the representation and contract, not a collection of legacy string-manipulation functions.

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char word[] = "Belize";

    printf("length = %zu\n", strlen(word));

    if (strcmp(word, "Belize") == 0) {
        puts("The strings are equal.");
    }

    return 0;
}
```

18. Compile and run the program.
19. Draw the bytes conceptually as B e l i z e \0.
20. Explain how strlen() knows where to stop even though you did not pass a length.

    It scans consecutive memory bytes until it encounters the null terminator byte (`\0`).

21. Change the comparison string and observe strcmp().

> **Security rule** **strlen() and strcmp() are appropriate only when their arguments are valid null-terminated C strings. They do not make an arbitrary byte buffer safe. In this course, do not treat strcpy(), strcat(), sprintf(), or similar legacy copying/concatenation patterns as default string-handling techniques.**

Do NOT deliberately call strlen() or strcmp() on an array that lacks a null terminator. That violates the function contract and invokes undefined behavior.

Part 6 - Separate interface from implementation

Create three files: stats.h, stats.c, and main.c. The header describes what another translation unit may call; the .c file contains the implementation.

stats.h

```c
#ifndef STATS_H
#define STATS_H

#include <stddef.h>

int sum_array(const int *values, size_t count);

#endif
```

stats.c

```c
#include "stats.h"

int sum_array(const int *values, size_t count)
{
    int total = 0;

    for (size_t i = 0; i < count; ++i) {
        total += values[i];
    }

    return total;
}
```

Write main.c yourself. It must include stats.h, create an array, compute its length, call sum_array(), and print the result.

Compile all source files together:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Wformat=2 -Wvla -Werror -g3 -O1 -fno-omit-frame-pointer \
    -fsanitize=address,undefined main.c stats.c -o stats_demo
```

Checkpoint C - Interface vs implementation

• Why does main.c include stats.h instead of stats.c?

Including `.c` files causes duplicate symbol definition errors during linking. Headers expose signatures without re-implementing code.

• What belongs in a header file?

Function prototypes, `struct` definitions, `typedefs`, and macros/constants.

• What problem do the include guards solve?

Prevents duplicate definition errors if a header is included multiple times in one file.

• Why is separate compilation useful once programs become larger?

Reduces re-compilation time and keeps modular code organized.

Part 7 - Structs and typedef

Create record.h and define the following type:

```
#ifndef RECORD_H
#define RECORD_H

typedef struct {
    int id;
    double value;
} Record;

#endif
```

In main.c:

22. Create one Record object using an initializer.
23. Print its fields using the . operator.
24. Create Record *record_ptr that points to the object.
25. Print the same fields through the pointer using ->.
26. Change value through the pointer and verify that the original object changed.

Interpretation **record.value means “select a field from this object.” record_ptr->value means “follow this pointer to an object, then select the field.”**

Part 8 - Dynamic memory and lifetime

Now create a Record whose lifetime is controlled explicitly rather than by the current block.

```
#include <stdio.h>
#include <stdlib.h>
#include "record.h"

int main(void)
{
    Record *record = malloc(sizeof *record);

    if (record == nullptr) {
        fputs("memory allocation failed\n", stderr);
        return EXIT_FAILURE;
    }

    record->id = 1;
    record->value = 42.5;

    printf("id=%d value=%.1f\n", record->id, record->value);

    free(record);

    return EXIT_SUCCESS;
}
```

27. Compile and run this program with the sanitizers enabled.

28. Explain why malloc() must be checked before dereferencing record.
    `malloc()` returns `nullptr` if memory allocation fails. Dereferencing `nullptr` crashes the program.

29. Explain why sizeof *record is preferable here to repeating sizeof(Record).
    Automatically stays correct if the pointer's underlying type changes later, preventing size mismatch bugs.

30. Temporarily comment out free(record), run the program, and observe whether your AddressSanitizer configuration reports the leak. Restore free(record) afterward. If your local sanitizer does not report leaks, record that observation rather than changing the code.

31. Explain what becomes invalid after free(record).

    The memory block at that address is returned to the OS; the pointer becomes a "dangling pointer."

Security rule Never dereference a pointer returned from an allocation function before checking for failure. Never use an object after its lifetime has ended. Every successful owned allocation needs a clear ownership and release plan. Assigning one pointer variable to nullptr does not make other aliases safe after free().

Part 9 - The bridge to linked data structures

Add this type to a new file named node.h:

```
#ifndef NODE_H
#define NODE_H

typedef struct Node {
    int value;
    struct Node *next;
} Node;

#endif
```

Do not build a linked list yet. Create only one dynamically allocated Node. Set value to 100 and next to nullptr, print value, then release the node.

> **Stop and reason** **The next field does not contain another Node. It contains an address at which another Node could exist. That single idea is the bridge from ordinary records to linked structures.**

• Why does the struct need the tag Node in struct Node *next?

Inside the `struct` body, the `typedef Record` alias isn't fully defined yet, so C requires `struct Node *` for self-reference.

• What does next == nullptr communicate?

Indicates this is the end of the chain and no further node exists.

• If next eventually points to another Node, where might that second Node live?

The second `Node` almost always lives dynamically allocated on the heap (via `malloc`), though it could technically be a local variable on the stack or sit in global/static memory.

• Who should be responsible for freeing dynamically allocated nodes?

The creator or owner of the data structure is responsible for freeing the memory, usually by calling a dedicated cleanup function like `free_list()` before the reference to the list is lost.

Part 10 - Build the program with Make

Your final folder should contain at least:

```
lab1/
├── Makefile
├── main.c
├── stats.c
├── stats.h
├── record.h
└── node.h
```

Create this Makefile. Recipe lines must begin with a TAB character, not spaces.

```
CC = gcc
CFLAGS = -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow -Wformat=2 -Wvla -Werror -g3 -O1
-fno-omit-frame-pointer
SANITIZERS = -fsanitize=address,undefined
TARGET = lab1
OBJECTS = main.o stats.o

.PHONY: all run clean

all: $(TARGET)

$(TARGET): $(OBJECTS)
	$(CC) $(OBJECTS) $(SANITIZERS) -o $(TARGET)

main.o: main.c stats.h record.h node.h
	$(CC) $(CFLAGS) $(SANITIZERS) -c main.c -o main.o

stats.o: stats.c stats.h
	$(CC) $(CFLAGS) $(SANITIZERS) -c stats.c -o stats.o

run: $(TARGET)
	./$(TARGET)

clean:
	rm -f $(TARGET) $(OBJECTS)
```

If you use Clang, run make CC=clang instead of editing the Makefile.

32. Run make clean.

33. Run make.
34. Run make run.
35. Fix every compiler or sanitizer diagnostic. Do not submit a warning-producing build.

Final integration task

Modify main.c so one execution demonstrates the concepts from the lab without becoming a data-structure implementation. Your final program must:

☐ Create an ordinary integer array and call sum_array().

☐ Create one Record object and access it through a pointer.

☐ Allocate one Record dynamically, check the result, use it, and free it.

☐ Allocate one Node dynamically, set next to nullptr, use it, and free it.

☐ Return EXIT_SUCCESS on success.

☐ Compile through the Makefile with zero warnings.

☐ Run without AddressSanitizer or UndefinedBehaviorSanitizer errors.

What not to use in this lab

Unless your instructor explicitly asks for them, do not introduce additional complexity. In particular, do not use:

- strcpy(), strcat(), sprintf(), gets(), or “safe because it has n in the name” substitutions without understanding their contracts;
- legacy null-pointer spellings such as NULL, integer 0, or (void *)0 in new code; use nullptr;
- variable-length arrays;
- pointer arithmetic beyond ordinary array indexing;
- macros beyond include guards;
- global mutable state;
- function pointers, unions, bit fields, or advanced preprocessor techniques.

Submission

Submit the complete lab1 folder or repository link according to your instructor’s normal submission method. Before submitting, verify all of the following:

☐ make clean && make completes successfully.

☐ The build produces no warnings.

☐ make run completes without sanitizer errors.

☐ All dynamically allocated memory in the final program has a clear matching free().

☐ All null pointers in new code are written using nullptr.

☐ No required compiler warning or sanitizer option was removed to hide a problem.

☐ Your source is organized into appropriate .h and .c files.

☐ You can verbally explain every pointer used in your final program.

Exit ticket - answer in your own words

1. What information does a pointer contain?

A raw memory address corresponding to a specific location in system memory.

2. Why does a function receiving an array usually also need a length?

In C, arrays decay into raw pointers when passed to functions, losing length information.

---

3. What makes a char array a valid C string?

It must terminate with a null byte (`'\0'`).

---

4. What is the difference between a struct object and a pointer to a struct object?

A struct object directly stores data fields; a pointer stores only the memory address where those fields reside.

---

5. What must happen after malloc() before you dereference the returned pointer?

Check if the pointer equals `nullptr` to handle allocation failures safely.

---

6. What does free() change about the lifetime of an allocated object?

It explicitly ends the dynamic allocation lifetime, making access to that memory invalid.

---

7. In a self-referential Node, what exactly is stored in next?

The memory address of another `Node` structure on the heap.

---

8. Why is "it printed the right answer once" not enough evidence that a C program is correct?

Undefined behavior can accidentally output expected results before causing memory corruption or crashes under different environments.
